LAPORAN PRATIKUM IoT II

Minggu 3



Dosen pengampu :

Dr. Ir. Septafiansyah Dwi putra, S.T.,M.T.

Agus ambarwari, S.Pd.,M.Kom.

Penulis :

Nama : Rheyadh Jhinan Mushaffa

NPM : 25782052

Kelas : TRI 3B

JURUSAN TEKNOLOGI INFORMASI

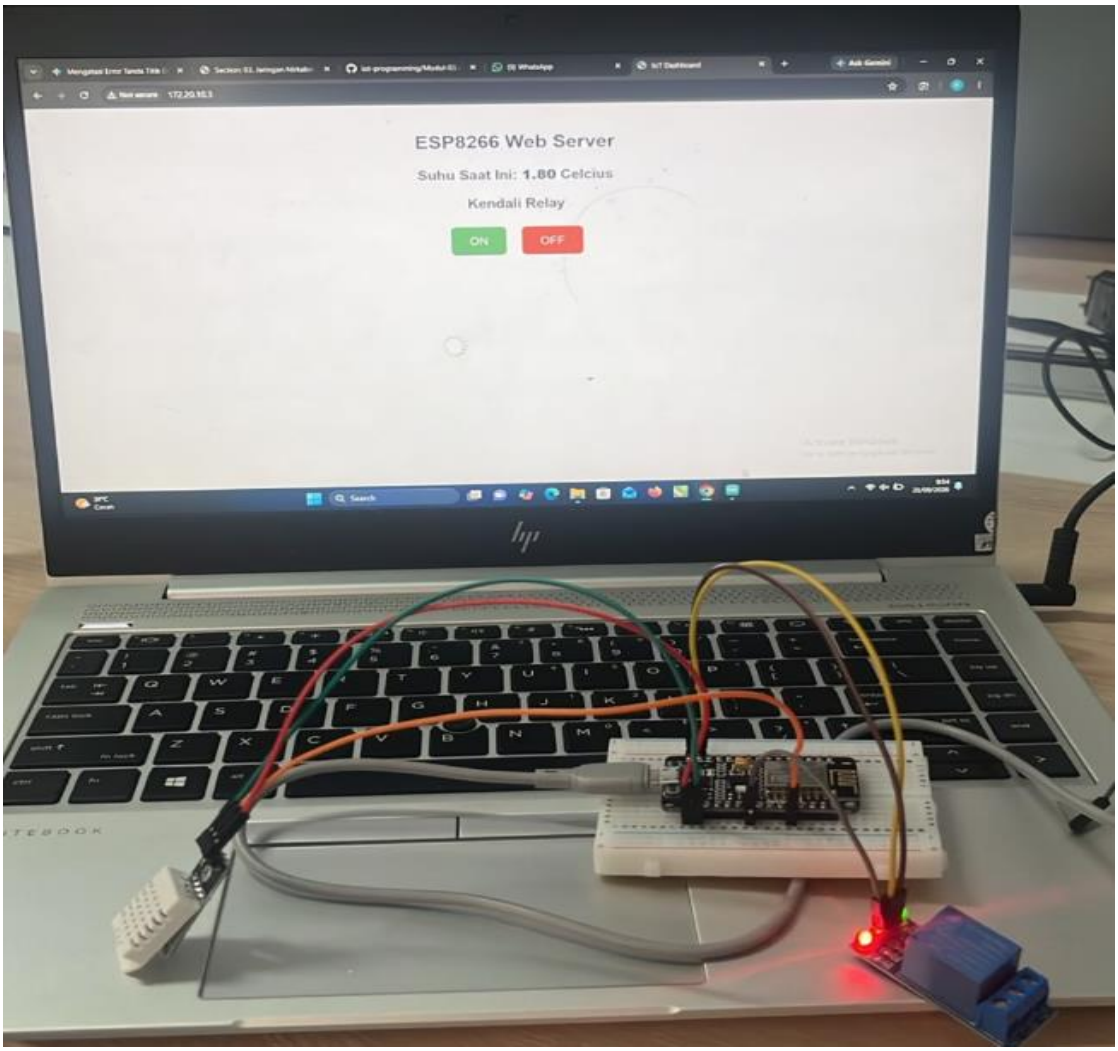
PROGRAM STUDI TEKNOLOGI

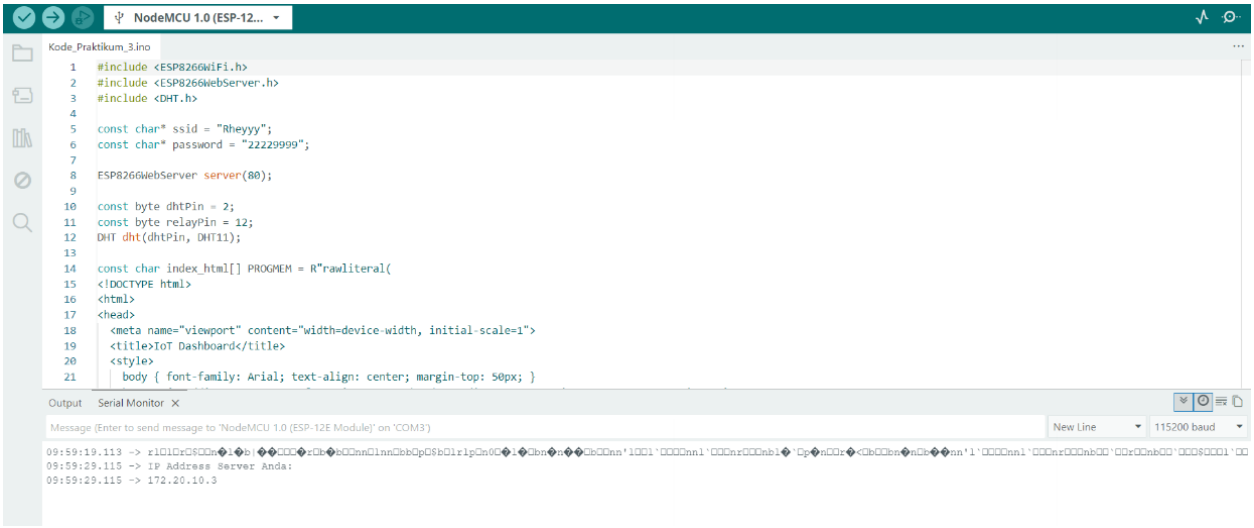
REKAYASA INTERNET

POLITEKNIK NEGERI LAMPUNG

TAHUN 2026

3.5.1 Praktikum 1: NodeMCU sebagai Web Server (Menyajikan HTML)





Langkah Kerja Rangkaian:

1. Hubungkan pin DATA / OUT / S DHT ke pin D4 (GPIO 2). (Pastikan VCC DHT ke 3V3, GND ke GND).

2. Hubungkan kaki Anoda (kaki panjang) LED ke pin D6 (GPIO 12). Hubungkan kaki Katoda (kaki pendek) LED ke salah satu kaki Resistor 220 Ohm, lalu hubungkan kaki resistor lainnya ke pin GND NodeMCU. (Catatan: LED ini dikendalikan oleh pin D6 / GPIO 12 yang pada kode program didefinisikan sebagai relayPin).

Penjelasan Singkat Kode:

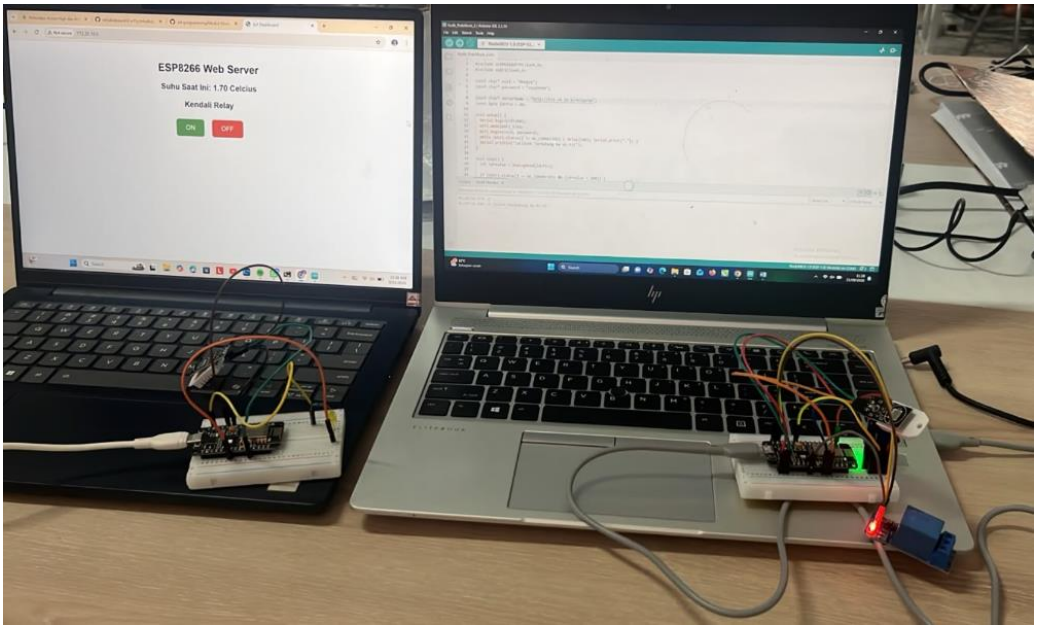
- ESP8266WebServer server(80);: Menginisialisasi library Server agar terus mendengarkan panggilan di jalur lalu lintas standar web (Port 80).

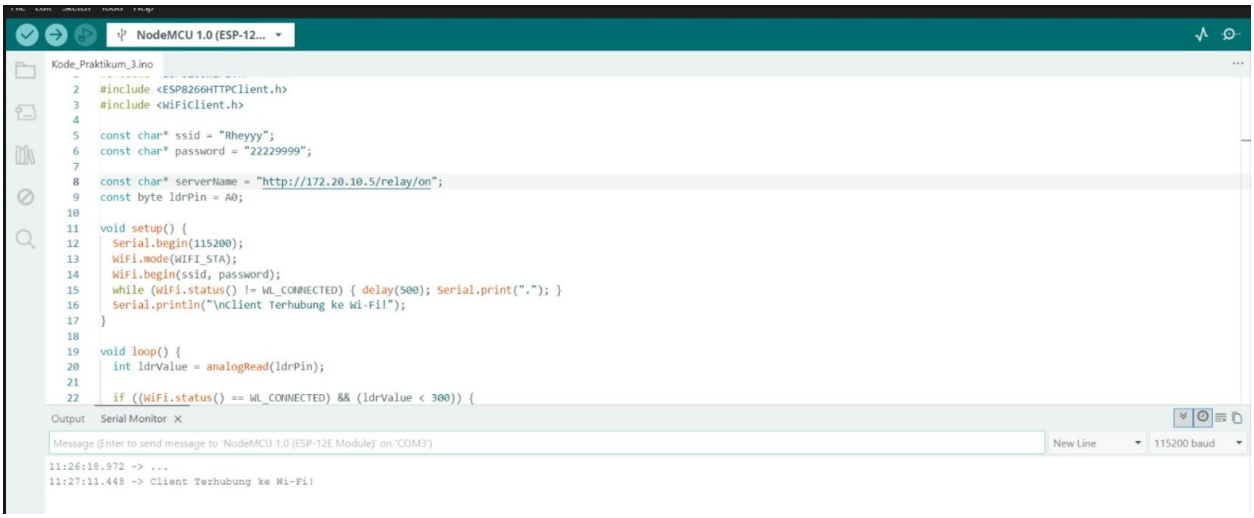
- html.replace("%TEMPERATURE%", String(t));: Fitur manipulasi teks (String). Perintah ini secara cerdas akan menyapu dokumen HTML, mencari teks placeholder %TEMPERATURE%, dan menggantinya dengan variabel angka suhu yang didapat dari sensor aktual sebelum dikirim ke layar pengunjung. Inilah esensi dari halaman web dinamis.

- server.on("/relay/on", handleRelayOn);: Fungsi Routing. Jika ada klien mengunjungi tautan URL berakhiran /relay/on, server akan menjalankan aksi mengubah state LED (menyalakan/mematikan output), lalu me-redirect layar pengunjung kembali ke halaman depan (Location: /) via kode 303.

- server.handleClient();: Berada di dalam loop() untuk terus-menerus mengecek apakah ada permintaan pengunjung yang datang setiap milidetiknya.

3.5.2 Praktikum 2: NodeMCU sebagai HTTP Client (Meminta Layanan)





Langkah Kerja Rangkaian:

1. Rangkai sensor LDR dan Resistor 10k Ohm sebagai pembagi tegangan (seperti di Modul 2) lalu hubungkan jalur sinyalnya ke pin A0.

Penjelasan Singkat Kode:

- WiFiClient client; dan HTTPClient http;: Kelas pemanggil untuk memulai eksekusi permintaan client jarak jauh (mirip fungsi aplikasi browser di laptop).
- http.begin(client, serverName);: Menargetkan sasaran URL server.
- int httpResponseCode = http.GET();: Mengeksekusi ketukan pintu (Request HTTP GET). Jika Server merespons sukses, nilai variabel ini akan berisi angka standar jaringan seperti 200 (OK) atau 303 (Redirect). Jika URL rusak atau Server offline, variabel akan mereturn nilai negatif.
- delay(10000);: Cooldown panjang 10 detik ini ditambahkan agar Client tidak memborbardir (flood/DDoS) Server dengan permintaan on secara terus-menerus setiap detik ketika kondisinya sedang gelap

3.6 Latihan

Latihan 1: Pemahaman Jaringan

Pertanyaan: Mengapa kita menggunakan mode jaringan Station (STA) pada kedua praktikum, dan bukan mode Access Point (AP)?

Jawaban:

- Terhubung ke Jaringan Bersama: Mode STA (*Station*) membuat NodeMCU bertindak sebagai *client* yang terhubung ke jaringan Wi-Fi yang sama (seperti *router* atau *hotspot* lokal). Hal ini memungkinkan mikrokontroler dan perangkat pengontrol (laptop/smartphone) berada dalam satu segmen jaringan lokal yang sama sehingga mudah saling berkomunikasi.
- Aksesibilitas dan Infrastruktur: Jika menggunakan mode AP (*Access Point*), NodeMCU akan memancarkan sinyal Wi-Fi sendiri. Akibatnya, perangkat yang terhubung ke NodeMCU akan kehilangan koneksi internet atau jaringan utama lokal, dan hanya bisa berkomunikasi secara terbatas dengan satu perangkat tersebut. Oleh karena itu, mode STA lebih ideal untuk sistem IoT berbasis infrastruktur jaringan yang ada.

Latihan 2: Modifikasi Antarmuka & Telemetri

Untuk menampilkan kelembapan dan membuat tombol *toggle* dinamis tunggal, berikut adalah modifikasi kode pada HTML dan fungsi handleRoot():



```
</head>
<body>
  <h1>ESP8266 Web Server</h1>
  <div class="sensor-box">
    <p>Suhu Saat Ini: <strong>%TEMPERATURE%</strong> Celcius</p>
    <p>Kelembapan: <strong>%HUMIDITY%</strong> %</p>
  </div>
  <h2>Kendali Relay</h2>
  %RELAY_BUTTON%
```

```
void handleRoot() {
  String html = index_html; // Salin kerangka HTML ke variabel dinamis
  float t = dht.readTemperature();
  float h = dht.readHumidity();

  // Mengganti teks placeholder %TEMPERATURE% dengan suhu nyata
  if (isnan(t)) {
    html.replace("%TEMPERATURE%", "--");
  } else {
    html.replace("%TEMPERATURE%", String(t));
  }

  // Mengganti teks placeholder %HUMIDITY% dengan kelembapan nyata
  if (isnan(h)) {
    html.replace("%HUMIDITY%", "--");
  } else {
    html.replace("%HUMIDITY%", String(h));
  }

  // Logika Tombol Dinamis Tunggal
  int relayState = digitalRead(relayPin);
  String buttonHtml = "";
  if (relayState == LOW) {
    buttonHtml = "<a href='/relay/on'><button class='btn-on'>NYALAKAN (ON)</button></a>";
  } else {
```

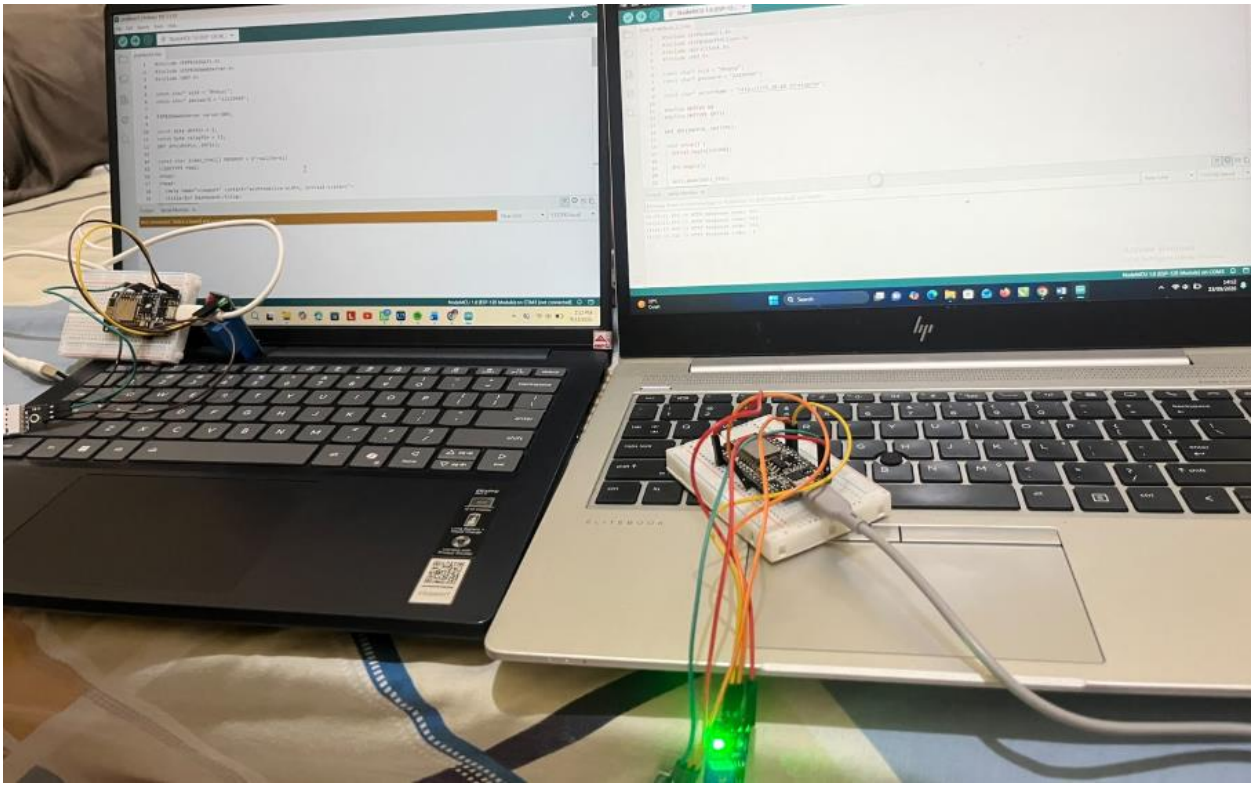
1. Penambahan Sensor Kelembapan (%HUMIDITY%)

- Konsep: Data kelembapan diambil dari sensor DHT11 menggunakan fungsi bawaan pustaka yaitu dht.readHumidity().
- Implementasi Kode:
    - Pada kerangka HTML (index_html), ditambahkan elemen penampung berupa placeholder %HUMIDITY% di dalam bagian kotak sensor ( ).
    - Di dalam fungsi handleRoot(), nilai pembacaan kelembapan dari sensor disimpan ke dalam variabel float h. Sebelum dikirim ke peramban (*browser*), dilakukan validasi menggunakan fungsi isnan(h) untuk memastikan sensor terbaca dengan benar (tidak mengalami kegagalan hubung).
    - Variabel teks placeholder %HUMIDITY% kemudian diganti menggunakan metode html.replace("%HUMIDITY%", String(h)) agar nilai kelembapan aktual tampil secara dinamis pada halaman web setiap kali halaman dimuat ulang.

2. Implementasi Tombol Dinamis Tunggal (*Toggle Button*)

- Konsep: Mengubah dua tombol terpisah (tombol ON dan tombol OFF) menjadi hanya satu tombol tunggal yang status tampilannya (warna dan teks) serta fungsi aksinya berubah secara adaptif mengikuti kondisi perangkat keras (status LED/Relay) saat itu.
- Implementasi Kode:
    - Tombol statis pada kerangka HTML dihapus dan diganti dengan placeholder %RELAY_BUTTON%.
    - Di dalam fungsi handleRoot(), status fisik pin relay dibaca terlebih dahulu menggunakan fungsi digitalRead(relayPin).
    - Melalui blok percabangan if-else:
        - Jika relay berstatus LOW (mati), kode secara dinamis menghasilkan elemen tautan HTML (<a href='/relay/on'>) dengan tombol berwarna hijau bertuliskan "NYALAKAN (ON)".
        - Jika relay berstatus HIGH (nyala), kode secara otomatis mengubah tampilan tautan (<a href='/relay/off'>) dengan tombol berwarna merah bertuliskan "MATIKAN (OFF)".
    - Placeholder %RELAY_BUTTON% kemudian diganti dengan string HTML tombol yang telah dikondisikan tersebut.

Latihan 3 (Analisis Kode Client): Pada Praktikum 2, *Serial Monitor Client* mencetak HTTP Response code. Analisislah apa yang akan dicetak oleh *Serial Monitor* Anda jika ternyata NodeMCU *Server* teman Anda mendadak kehilangan daya (*offline*)? (Silakan cabut daya Server dan lihat hasil erornya).





Hasil Analisis: Pengujian ini dilakukan dengan menjalankan NodeMCU Client dan NodeMCU Server untuk memastikan keduanya dapat berkomunikasi. Setelah itu, NodeMCU Server dimatikan dengan cara mencabut kabel USB atau sumber daya yang terhubung ke Server, sedangkan NodeMCU Client tetap menyala. Selanjutnya Client mencoba mengirim GET Request ke Server yang sudah dalam keadaan offline. Berdasarkan hasil pengujian pada Serial Monitor, Client tetap berhasil terhubung ke Wi-Fi dan menampilkan pesan “Client Terhubung ke Wi-Fi!”, tetapi ketika mencoba berkomunikasi dengan Server, muncul “HTTP Response code:

-11". Hal ini menunjukkan bahwa Client tidak mendapatkan respons HTTP dari Server karena Server dalam keadaan offline.

Latihan 4 (Pengembangan Algoritma): Anda ingin mendesain Klien Pendeteksi Suhu. Modifikasi *conditional block* if di Praktikum 2 sehingga *Client* mengirim permintaan (GET Request) HANYA JIKA sensor DHT membaca suhu melebihi 35 Celcius (bukan menggunakan LDR). Tulis baris algoritmanya.

```
32  void loop() {
33    float temperature = dht.readTemperature();
34
35    Serial.print("Suhu: ");
36    Serial.print(temperature);
37    Serial.println(" C");
38
39    if ((WiFi.status() == WL_CONNECTED) && (temperature > 35)) {
40      WiFiClient client;
41      HTTPClient http;
42
43      http.begin(client, serverName);
44
45      int httpResponseCode = http.GET();
46
47      Serial.print("HTTP Response code: ");
48      Serial.println(httpResponseCode);
49
50      http.end();
51
52      delay(10000);
53    }
54
55    delay(2000);
56  }
```

Pada Latihan 4, sensor LDR pada Praktikum 2 diganti dengan sensor DHT22 untuk membaca suhu. Library DHT.h ditambahkan agar NodeMCU dapat menggunakan sensor DHT22. Kemudian dht.readTemperature() digunakan untuk mengambil nilai suhu. Program selanjutnya menggunakan kondisi: temperature > 35 Artinya, GET Request hanya dikirim ke Server jika suhu lebih dari 35°C. Jika suhu masih 35°C atau kurang, Client tidak mengirim request ke Server. Dengan demikian,

pengiriman perintah ke Server sekarang dikendalikan berdasarkan nilai suhu dari sensor DHT22, bukan lagi berdasarkan nilai LDR.

3.7 Tugas: Pengembangan Web Server Cerdas dan Analisis Arsitektur

